

Forrageamento e Comunicação em Enxames Simulação de Formigueiro

Disciplina: Inteligência Artificial I

Autor: Luciano Magri

IFRS — Campus Farroupilha

Abril de 2026

1. Objetivo e Contexto

Modelar o comportamento emergente de um enxame de formigas buscando alimento em um ambiente desconhecido. Cada formiga é um agente autônomo que:

- Executa caminhada aleatória com limite de energia (20 passos).
- Marca caminhos fracassados como restrições globais (feromônio negativo).
- Compartilha rotas de sucesso com todo o enxame ao encontrar alimento.
- Adapta seu comportamento conforme aprende com os peers.

O objetivo é demonstrar como comunicação síncrona e estado global compartilhado podem coordenar agentes para resolver coletivamente um problema que seria inviável de forma isolada.

2. Arquitetura do Sistema

2.1 Estrutura de Arquivos

- `base_formiga.py`: Classe `Formiga` com lógica de movimentação e comunicação.
- `formiga.py`: Classe `Ambiente` e instanciação de agentes.
- `executor_formigueiro.py`: Orquestrador que monitora o estado global.

2.2 Componentes Principais

- Classe `Ambiente`:

Mantém a grade $N \times N$, posição do alimento (oculta dos agentes), contador de coletas, e lista global de caminhos inválidos (feromônio negativo). Oferece interfaces para verificação de alimento e obtenção de vizinhos válidos.

- Classe `Formiga`:

Agente autônomo com `inbox` (`asyncio.Queue`) para recebimento de mensagens. Mantém histórico da rota atual, rota conhecida (broadcast recebido), e posição. A corrotina `agir()` executa tentativas infinitas de forrageamento.

- Orquestrador (`main`):

Cria corrotinas para cada formiga e monitora o contador de coletas de forma passiva. Não dita rotas; apenas cancela todas as tarefas quando `alimento_coletado == 4`.

3. Implementação das Tarefas (A, B, C, D)

Tarefa A: Movimentação Assíncrona

A corrotina `_forragear()` executa a busca em um loop de até 20 passos:

```

for passo in range(MAX_PASSOS):
    await self._processar_inbox() # verifica mensagens
    if self.known_food_path: # segue rota conhecida se disponível
        next_pos = self.known_food_path[passo + 1]
    else:
        next_pos = random.choice(self._vizinhos_validos())
    self.position = next_pos
    self.route.append(next_pos)
    if self.ambiente.tem_comida(self.position):
        await self._retornar_sucesso(); return
    await asyncio.sleep(0.02)

```

O método `_vizinhos_validos()` retorna vizinhos 4-adjacentes dentro da grade, evitando aqueles que aparecem em caminhos inválidos conhecidos (Tarefa B).

Tarefa B: Reforço Negativo (Feromônio)

Ao atingir 20 passos sem alimento, `_retornar_fracasso()` executa:

- Registra a rota completa em `ambiente.caminhos_invalidos` (estado global).
- Envia broadcast do tipo 'fracasso' com a rota para todas as outras formigas.
- Formigas receptoras adicionam a rota à lista global e passam a evitar o próximo passo daquele caminho ao escolher vizinhos.

Este mecanismo é análogo ao Nogood do algoritmo Asynchronous Backtracking (ABT), onde um agente comunica aos demais uma atribuição (rota) que não leva a solução válida.

Tarefa C: Comunicação de Sucesso

Ao encontrar alimento, `_retornar_sucesso()` executa:

- A própria formiga memoriza a rota (`self.known_food_path = rota`).
- Incrementa o contador global (`ambiente.alimento_coletado`).
- Envia broadcast do tipo 'sucesso' com a rota exata para todas as demais formigas.
- Formigas receptoras armazenam a rota e passam a seguir diretamente nas próximas tentativas, sem exploração aleatória.

Este padrão é inspirado no algoritmo LRTA(n)*, onde agentes exploram um ambiente e compartilham as melhores rotas encontradas para acelerar convergência coletiva.

Tarefa D: Orquestrador

O executor cria uma corrotina por formiga e monitora passivamente:

```

while ambiente.alimento_coletado < ambiente.capacidade_maxima:
    if timeout_exceeded:
        break
    await asyncio.sleep(0.1)

```

Quando `alimento_coletado` atinge 4 (ou timeout), todas as tarefas são canceladas graciosamente com `tarefa.cancel()`.

4. Análise

4.1 Exploração vs. Exploração

O sistema exibe transição clara entre dois modos operacionais:

Fase de Exploração: Antes de qualquer formiga encontrar alimento, o enxame opera em modo de exploração pura — cada agente executa passeio aleatório independente, cobrindo regiões diferentes de forma emergente.

Fase de Exploração: Uma vez que uma formiga bem-sucedida compartilha a rota, o sistema faz transição coletiva para exploração — todas as demais formigas absorvem o conhecimento via inbox e passam a seguir o caminho conhecido diretamente, sem desvio.

Comparação com aprendizado isolado: um agente isolado precisaria redescobrir o alimento a cada tentativa individual. No sistema coletivo, o custo de exploração é amortizado — uma única descoberta beneficia imediatamente todo o enxame. Isso é análogo ao reforço positivo de feromônios em formigas reais: a trilha bem-sucedida é reforçada (aqui, pela rota compartilhada), convergindo comportamento do grupo para a solução ótima sem que cada indivíduo aprenda isoladamente.

4.2 Impacto do Reforço Negativo

O feromônio negativo (lista `caminhos_invalidos`) atua como poda do espaço de busca. A cada tentativa fracassada, um subconjunto de arestas do grafo é marcado como improdutivo. Nos logs, é observável que após tentativas iniciais fracassadas, formigas seguintes desviam de direções já exploradas e exploram regiões novas, reduzindo redundância significativamente.

Limitação de escala: Em mapas muito grandes (ex.: 50×50), o número de caminhos possíveis de até 20 passos cresce exponencialmente ($\sim 4^{20} \approx 10^{12}$). Bloquear apenas rotas percorridas não garante cobertura suficiente, e alimento pode nunca ser encontrado. Estratégias complementares seriam necessárias: aumentar limite dinâmico, usar busca em leque com múltiplas colônias, ou combinar feromônio negativo com feromônio positivo de exploração para direcionar ativamente formigas a regiões não visitadas.

4.3 Comunicação Direta vs. Estigmergia

Implementação atual usa passagem direta de mensagens (`asyncio.Queue`). Na natureza, formigas alteram o ambiente depositando feromônios — comunicação indireta (estigmergia).

Para implementar stigmergia, seriam necessárias duas mudanças principais:

- Mapa de feromônios em Ambiente: `dict[tuple, float]` inicializado em 0.0. Formigas bem-sucedidas depositam feromônio ao longo da rota (incremento proporcional). Orquestrador aplica evaporação periódica multiplicando valores por fator $\rho < 1$.

- Seleção probabilística em Formiga: Substituir `random.choice()` por roleta (roulette wheel) baseada em valores de feromônio dos vizinhos (maior feromônio = maior probabilidade). Remover completamente `_broadcast()` e `_processar_inbox()`.

Com essas mudanças, nenhum agente conhece diretamente decisão de outro; coordenação emerge apenas de modificação e leitura do estado do ambiente — princípio fundamental de estigmergia.

5. Resultados e Observações

Simulações executadas confirmam comportamento esperado:

- Convergência: Alimento é descoberto e coletado por 4 formigas em menos de 30 segundos.
- Transição de exploração para exploração: Visível nos logs — primeiras tentativas mostram modo 'aleatório', após broadcast há mudança abrupta para 'rota conhecida'.
- Eficiência coletiva: Uma descoberta beneficia imediatamente todo o enxame; não há redescoberta individual.
- Feromônio negativo: Bloqueios de caminhos aparecem nos logs; ants demonstram seletividade após registro de rotas fracassadas.

6. Conclusão

A simulação de formigueiro implementa, de forma didática mas funcional, os dois pilares do Ant Colony Optimization: evitar soluções ruins (feromônio negativo, análogo a *Nogood*) e reforçar as boas (broadcast de sucesso, análogo a *LRTA(n)**). O sistema demonstra como mecanismos simples de comunicação assíncrona podem gerar comportamento emergente complexo e eficiente em solução de problemas distribuídos.

Enquanto a comunicação direta por mensagens funciona bem em escala pequena, a migração para estigmergia (comunicação indireta via modificação de ambiente) seria essencial para escalabilidade em cenários reais, eliminando necessidade de conhecimento global e permitindo operação totalmente distribuída.